

I. Overview of LLM Evaluations

We will begin by defining the scope of what we are studying, what **LLM Evaluations (Evals)** are, and the different categories they fall into.

- **Model Evaluations (Model Evals):** These evaluations measure the performance of the underlying foundational Large Language Model (LLM) itself. We will not focus on this category here.
 - **Application Evaluations (Application Evals):** These evaluations focus on measuring the performance of an entire LLM-based system or workflow built for a specific application. Our discussion will focus entirely on **Application Evals**, as this is where we will be focusing our efforts.
-

II. Case Study: Automating Customer Email Routing at Zomato

To understand how to evaluate an LLM application, we will look at a simple, real-world scenario.

Imagine we are software engineers working at a company like **Zomato**. Because we are a large company with many customers, we receive a massive volume of emails daily. Manually reading and replying to every single email is extremely difficult and inefficient. To automate this setup, we want to build a system that reads the email content and classifies it.

Based on the content, the email must be classified into one of three categories:

1. **Billing Related Queries**
2. **Technical Problems**
3. **General Queries**

Once classified, we can automatically route the email to the appropriate team:

- **Billing Related Queries** are routed to our **Billing Team**.
- **Technical Problems** are routed to our **Technical Team**.
- **General Queries** are routed to our **Customer Support Team**.

This automation ensures we do not need a human agent to manually read emails and tag teams, saving significant time and resources.

To implement this, we build a simple LLM-based application. We set up an **LLM customer agent** and provide it with a prompt instructing it to read incoming emails and decide where to route them.

Although this application can be built in just 5 to 10 minutes, we cannot directly deploy it to production. Before deployment, we must rigorously evaluate the system to ensure its reliability. This evaluation workflow is identical whether we are testing a simple application like this or a highly complex agentic system.

III. The Core LLM Application Evaluation Workflow

The complete, repeatable workflow for evaluating an LLM-based application consists of the following key steps:

Step 1: Define the Task and Target

- **Target:** The target is the entire workflow or system we want to evaluate—in this case, our email classification and routing application.
- **Task:** The specific task we want to evaluate is classification. We need to check whether our system can accurately classify and route incoming emails.

Step 2: Define the Success Criteria & Metrics

- We must establish how we will know if the system is working correctly. For a classification task, our success criterion is classification performance, and our metric is **Accuracy**.
- **Accuracy** is calculated as the ratio of correct classifications to total inputs. For example, if the system receives 100 queries and routes 90 of them correctly, we say the system has an **Accuracy** of 90%.

Step 3: Build a Golden Dataset

- We must build a dataset to evaluate our application. This dataset will contain two main columns:
 - **Message:** The actual text or email content.
 - **Expected Type/Label:** The correct, manually verified classification category.
- Examples of entries in this dataset:
 - "My card was charged twice" (→) **Billing Issue**
 - "The app crashes on login" (→) **Technical Issue**
 - "What are your hours?" (→) **General Issue**
- **Golden Dataset:** This manually curated and labeled dataset is called the **Golden Dataset** in LLM evaluations.
- **Dataset Size:** Typically, a Golden Dataset should contain between **50 to 500 rows** of data.
- **How to Build It:** The best way to build this dataset is to collect actual historical data (such as past customer chats or emails) and have someone manually label each entry.

Step 4: Define the Evaluation Method

- We must decide who or what will perform the evaluation. We have three main options:
 1. **Automated (Code-Based/Deterministic) Evaluation:** In this method, we write simple code (like a Python script) to compare the LLM's outputs with the expected labels and

calculate the metric (e.g., accuracy). This is ideal for simple classification tasks where we compare short, predefined labels.

2. **Human Evaluation:** Humans manually review and score the outputs. While highly accurate for subjective tasks, hiring humans is very costly and slow.
3. **LLM-as-a-Judge:** We use a second, separate LLM to evaluate our application's outputs. This method is the middle ground between automated code and human evaluation. It is essential for chatbot applications that generate long paragraphs of text. We cannot easily write deterministic code to compare two long text paragraphs because exact matches fail to capture semantic meaning, but an LLM can evaluate whether the semantic meaning matches.

Step 5: Run the Model

- We feed the messages from our **Golden Dataset** into our LLM application to generate routing predictions for each message.

Step 6: Evaluate the Results

- We run our evaluation method (e.g., our Python script) to compare the LLM's predictions against the expected labels in our dataset. This calculates our final evaluation score. For example, we might find that our system achieves an initial **Accuracy** of 80%, meaning it classified 80 out of 100 emails correctly and made mistakes on 20.

Step 7: Analyze the Results

- When our system makes errors, we must analyze the mistakes to understand where the system is failing and identify the scope of improvement.
- Key areas of improvement to analyze:
 - **System Prompt:** Our prompt instructions might be ambiguous, causing the model to confuse similar categories (e.g., billing and technical queries).
 - **Model Selection:** The underlying LLM we chose might be too weak or have too few parameters (such as a smaller, open-source model) to handle the task.

Step 8: Improve the System and Iterate

- Based on our analysis, we make changes to improve the system, such as tweaking the system prompt or swapping the model for a heavier, more capable LLM.
- **Repeatability:** LLM evaluations must be repeatable. After updating the system, we run it again on the exact same **Golden Dataset** and calculate the new score. For example:
 - First run: 80% accuracy.
 - We tweak the prompt and re-run: accuracy improves to 90%.
 - We swap to a larger, heavier LLM and re-run: accuracy improves to 95%.

- We continue this iterative loop of running, evaluating, analyzing, and improving until we are satisfied with the performance.

Step 9: Deploy the System

- Once our system reaches a reliable performance level (e.g., 95% accuracy) and we are satisfied, we deploy the system online.

Step 10: Monitor in Production

- Even after a successful deployment, our job is not done. We must continuously monitor the system because it can still fail when encountering real, unseen production data from customers.

Step 11: Implement the Feedback Loop

- When a failure occurs in production (e.g., an email that should have been labeled "Billing" is misrouted to the technical team), we must capture it.
 - **Production Failure Handling:** In production, we set up a process where downstream teams (like the technical team) can flag misrouted emails. Once flagged, we extract the content of that specific email and add it back into our **Golden Dataset**.
 - This continuous feedback loop enriches our **Golden Dataset** over time, making it more robust. We then restart the entire evaluation workflow on the updated dataset to further improve our system. This loop runs continuously for as long as the system is deployed in production.
-

IV. Multi-Eval Architecture for Single Applications

A critical concept we must remember is that **a single LLM-based application may run several different LLM evaluations simultaneously**.

While our simple email classification example used only a single evaluation (accuracy of classification), complex applications require multiple specialized evals. For example, in a **Retrieval-Augmented Generation (RAG)** application, we might run separate, concurrent evaluations for:

1. **Retriever Performance:** Measuring how well the system retrieves relevant documents.
2. **Embedding Model Performance:** Evaluating the quality of the vector representations used.
3. **Complete RAG Workflow:** Testing the end-to-end performance of the retrieval and generation loop.
4. **System Latency:** Evaluating how fast the entire system responds.

Therefore, running multiple concurrent evaluations is a standard practice in production LLM application development.